



**UNIVERSIDAD AUTÓNOMA
DE SAN LUIS POTOSÍ**



Facultad de Ingeniería

Área de Ciencias de la Computación

Alumno: Lored Huerta Jesús Emmanuel

Materia: Administración de Bases de Datos

Profesora: Claudia Alicia Méndez Hernández

Tema: Práctica 2

Fecha: 05/03/26

Semestre: 2025-2026/II

Grupo: 231301

Índice

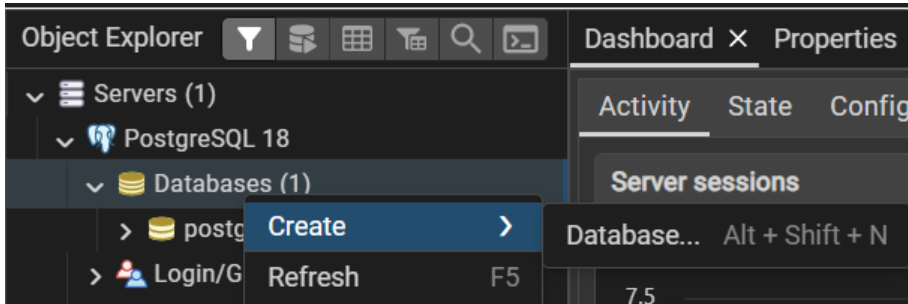
Creación de la Base de Datos en PostgreSQL	3
Creación del proyecto en Apache NetBeans	5
Instalar driver para conectar la base de datos.....	6
Creación de clase conexión	7
Creación del formulario Terminal	9
Diseño.....	9
Código	13
Script completo de la base de datos	21

Creación de la Base de Datos en PostgreSQL

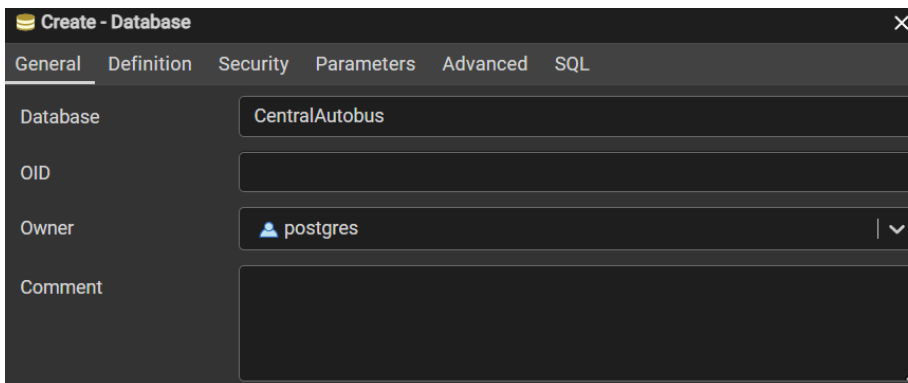
Abrimos el programa PgAdmin4, si nos pide contraseña iniciamos sesión (debería ser 'postgres' si se siguió las instrucciones de instalación del archivo

https://didactic.uaslp.mx/pluginfile.php/460640/mod_resource/content/9/Instalaci%C3%B3nPostgreSQLNetbeans.pdf.

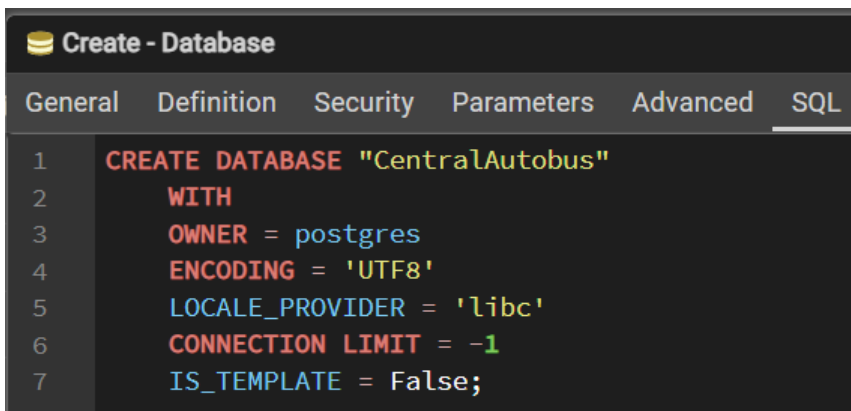
Después de iniciar sesión, en el menú lateral buscamos la opción que dice "Servers/PostgreSQL 18/Databases", le damos clic derecho y seleccionamos la opción de Create->Database



Escribimos el nombre de la base que será "CentralAutobus".

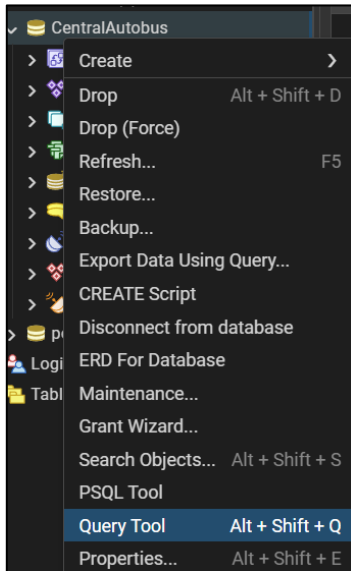


Nos vamos a la pestaña que dice "SQL" y copiamos la sentencia que crea nuestra base de datos para guardarla en nuestro Script.



Damos clic en “Save” y se creará la base de datos.

La ubicamos en el menú lateral, le damos clic derecho y seleccionamos la opción “Query Tool”, se nos abrirá una pestaña y podremos empezar a escribir nuestro script ahí.



Empezaremos con la creación de nuestros dos esquemas:

```
CREATE SCHEMA Atencion_Cliente;
```

```
CREATE SCHEMA Logistica;
```

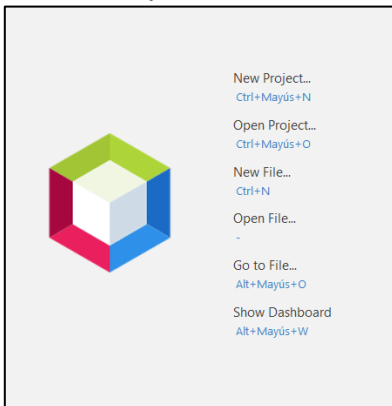
Y crearemos la tabla con la que vamos a trabajar en el formulario (el script completo de la base de datos se encontrará en la última sección de este documento).

```
CREATE TABLE Logistica.Terminal
```

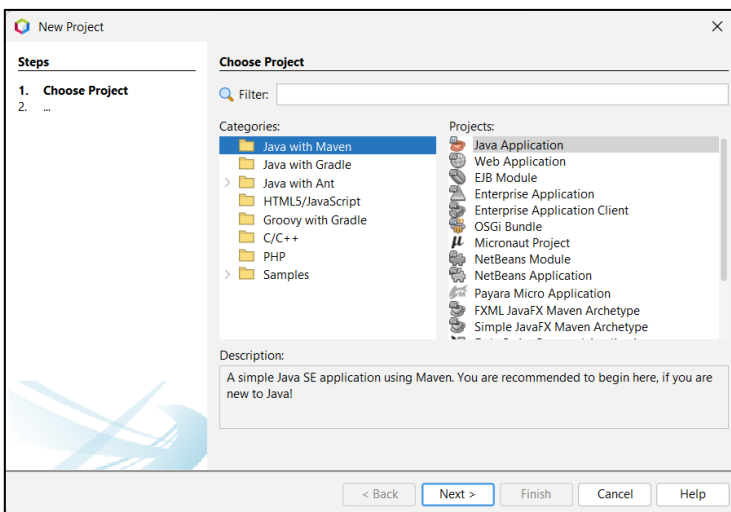
```
(  
    IdTerminal BIGSERIAL NOT NULL,  
    CodigoTerminal VARCHAR(10) NOT NULL,  
    NombreCiudad VARCHAR(100) NOT NULL,  
    NumeroAndenes INT NOT NULL,  
  
    CONSTRAINT PK_TERMINAL PRIMARY KEY(IdTerminal),  
    CONSTRAINT UQ_CODIGO_TERMINAL UNIQUE(CodigoTerminal)  
);
```

Creación del proyecto en Apache NetBeans

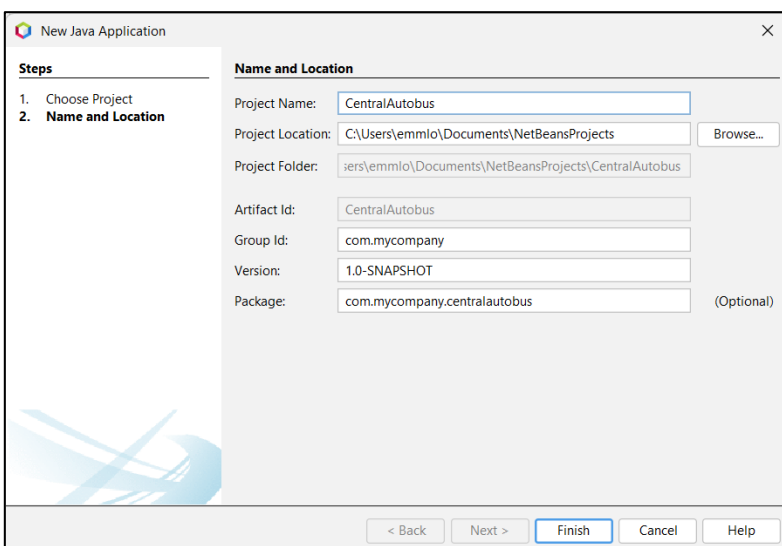
Abrimos Apache NetBeans y seleccionamos la opción de “New Project..”



Seleccionamos “Java with Maven” y “Java Application”.

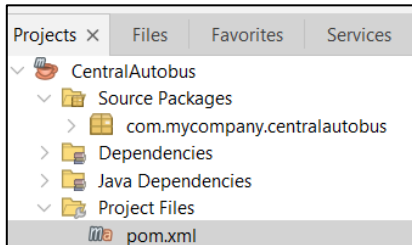


Nombramos el proyecto, en este caso “CentralAutobus” y le damos clic en Finish.



Instalar driver para conectar la base de datos

Nos vamos nuevamente al menú lateral, en Projects buscamos la carpeta llamada "Project Files" y seleccionamos el archivo pom.xml que se encuentra dentro de ella.



Dentro de ese archivo, buscamos la etiqueta que dice "<packaging>" y escribiremos el siguiente fragmento de código debajo de ella, para que, una vez guardado los cambios en el archivo, se descargue el driver necesario dentro de la carpeta "Dependencies".

```
<dependencies>

  <!-- Source:https://mvnrepository.com/artifact/org.postgresql/postgresql -->

  <dependency>

    <groupId>org.postgresql</groupId>

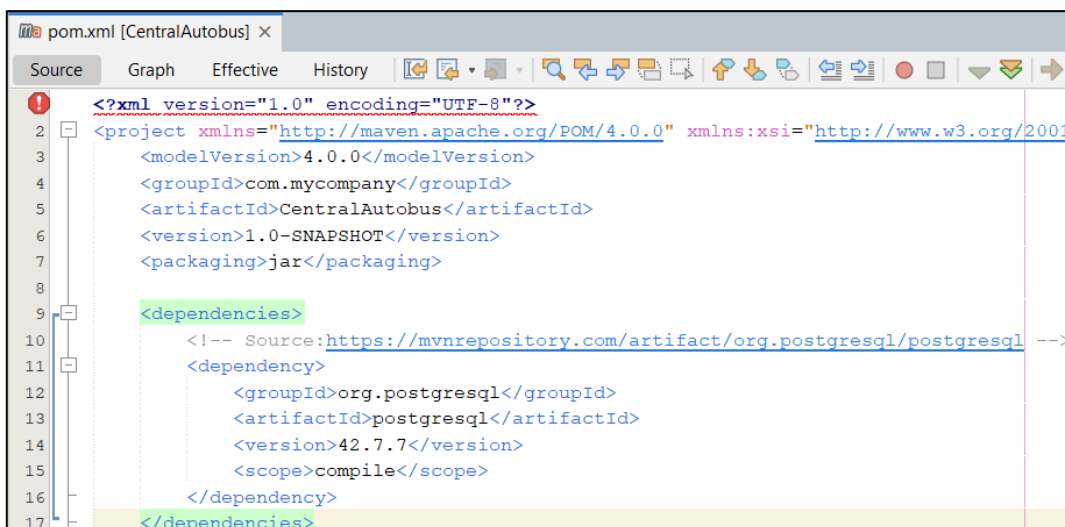
    <artifactId>postgresql</artifactId>

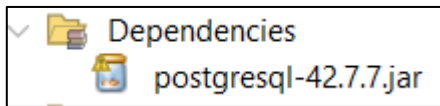
    <version>42.7.7</version>

    <scope>compile</scope>

  </dependency>

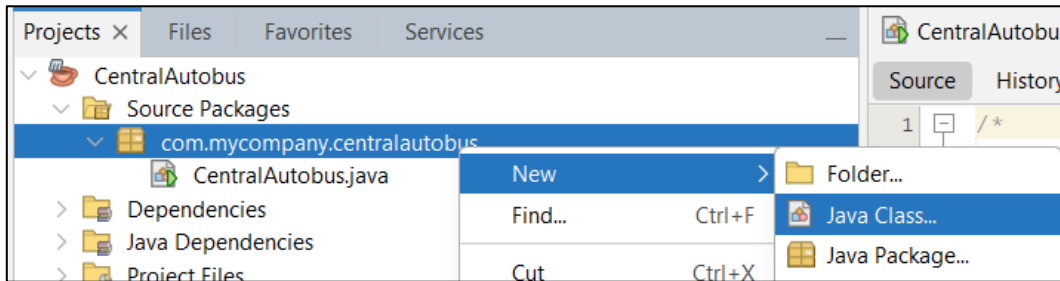
</dependencies>
```



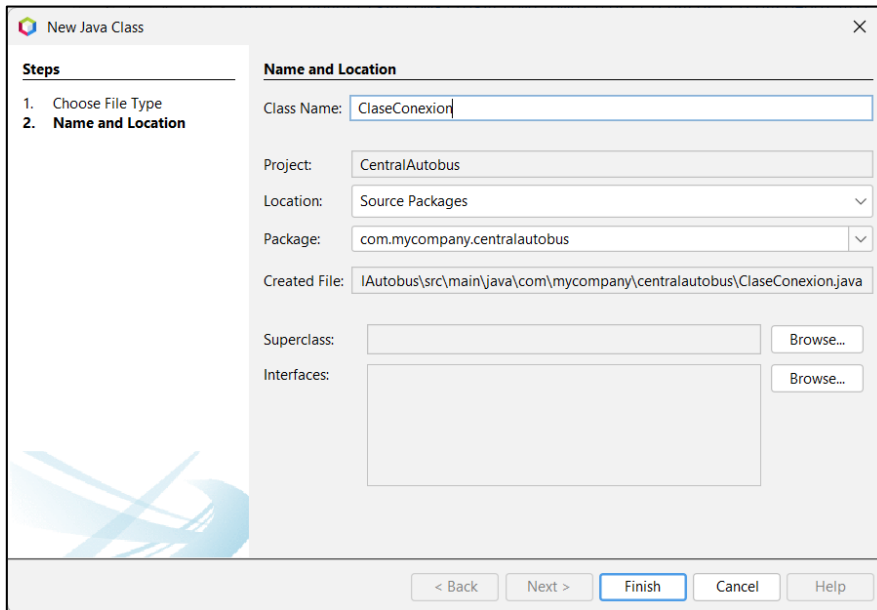


Creación de clase conexión

En el menú lateral, nos vamos a Projects/CentralAutobus/Source Packages/com.mycompany.centralautobus y le damos clic derecho -> New -> Java Class...



Le damos el nombre de "ClaseConexion" y aceptamos.



Dentro de la clase, declararemos las siguientes variables y el método conexión, que son necesarios para nuestra conexión:

```
package com.mycompany.centralautobus;
```

```
import java.sql.Connection;
```

```
import java.sql.DriverManager;
```

```
import javax.swing.JOptionPane;
```

```

public class ClaseConexion {
    private Connection conectar = null;
    private String user = "postgres";
    private String password = "postgres";
    private String bd = "CentralAutobus";
    private String ip = "localhost";
    private String puerto = "5432";
    private String cadena = "jdbc:postgresql://" + ip + ":" + puerto + "/" + bd;

    public Connection establecerConexion(){
        try{
            Class.forName("org.postgresql.Driver");
            this.conectar = DriverManager.getConnection(cadena, user, password);
        }catch (Exception e){
            JOptionPane.showMessageDialog(null, "Error: " + e.toString());
        }
        return this.conectar;
    }
}

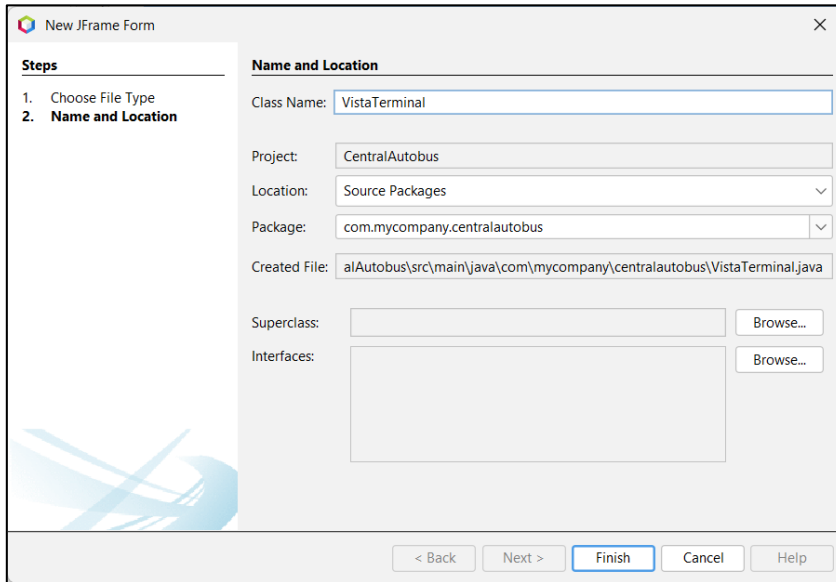
```


Creación del formulario Terminal

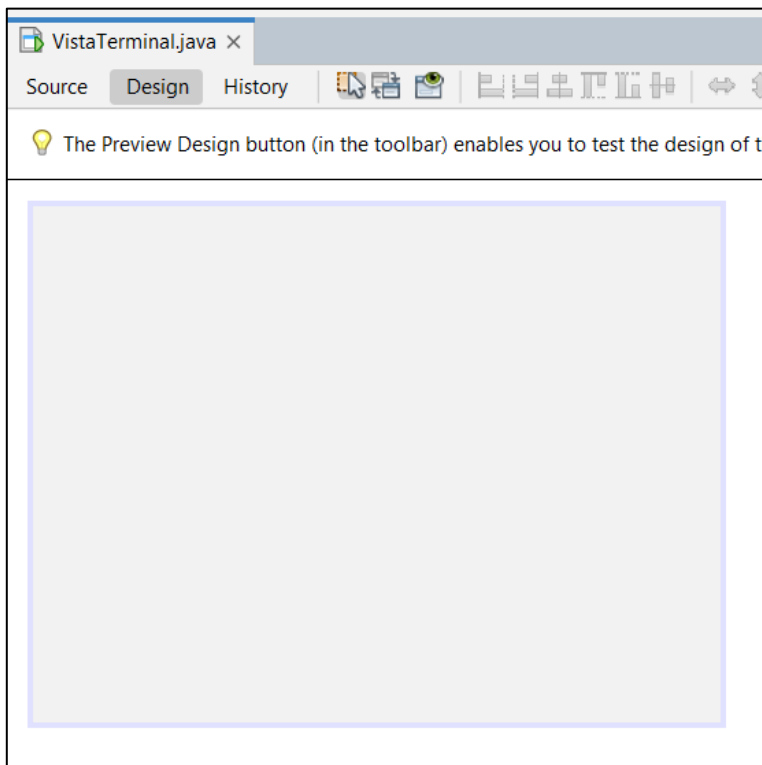
Diseño

Regresamos al paquete “com.mycompany...”, le damos clic derecho -> New -> JFrame Form...

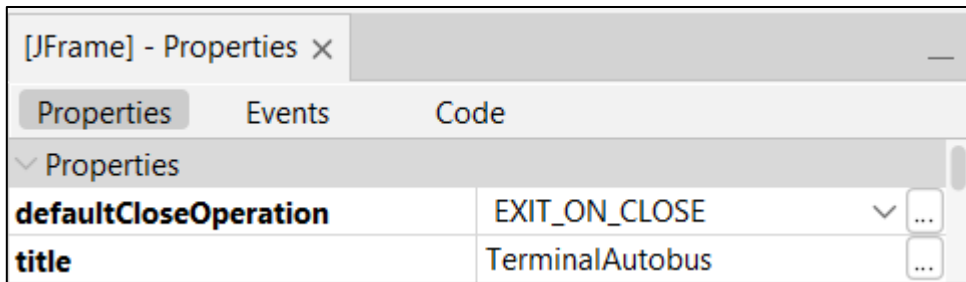
Le pondremos de nombre “Vista Terminal”.



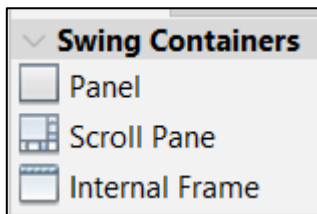
Y se nos creará el siguiente formulario vacío:



Hacemos clic en el frame y le cambiamos el title.

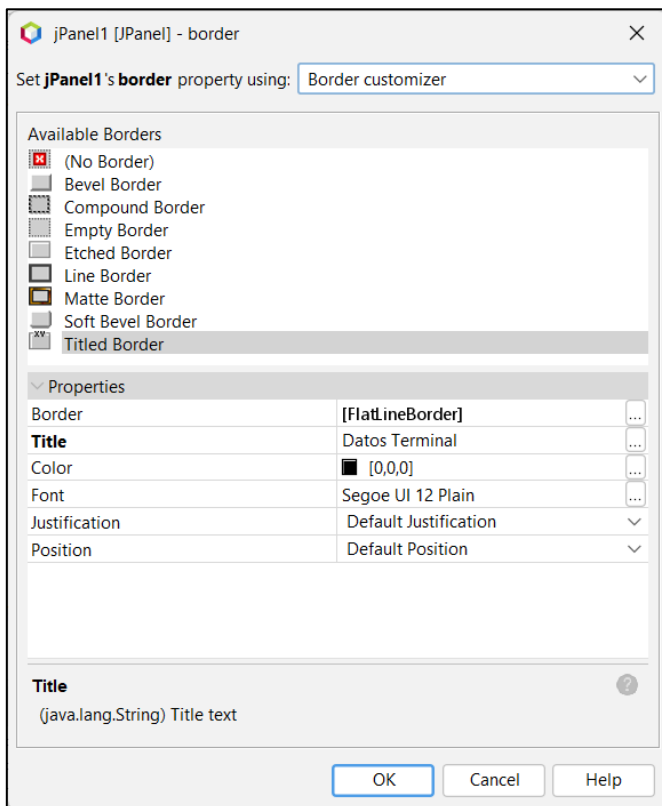


Agregaremos 2 panel:



Y le daremos clic a uno, nos vamos a las propiedades y donde dice “border” le damos clic.

Seleccionamos Titled border y donde dice Title escribimos “Datos terminal”.

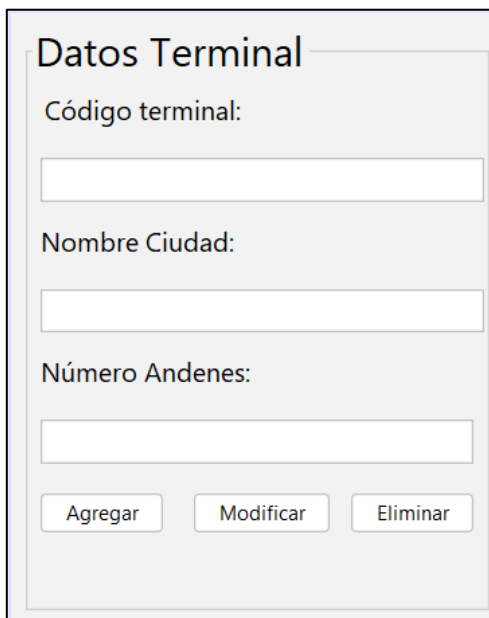


Haremos lo mismo con el otro, escribiendo “Lista terminal” y se debería de ver así nuestro formulario.



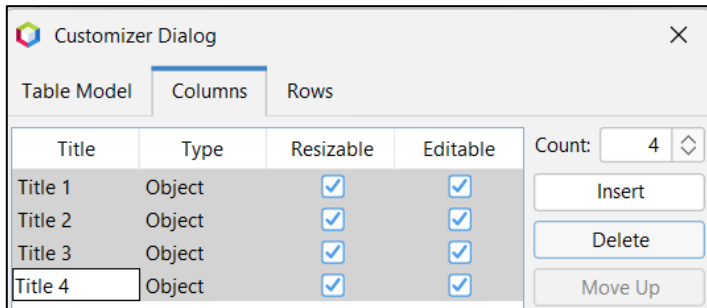
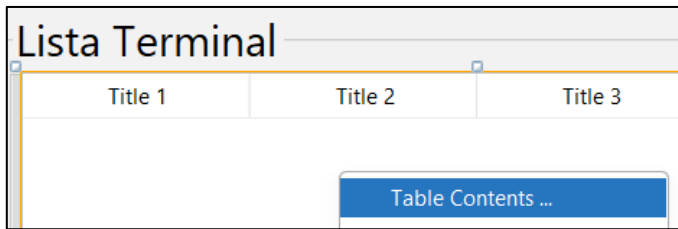
Dentro del panel de “Datos Terminal” Agregaremos 3 label, 3 text field y 3 button.

Y modificaremos sus atributos más importantes como el text y Font, estos se encuentran en el panel de propiedades.

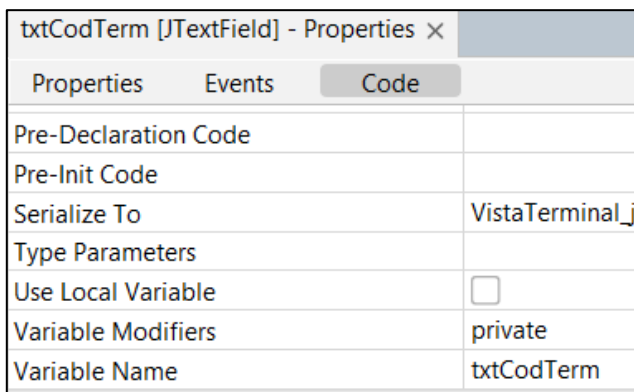


En el panel “Lista Terminal” agregaremos una Table.

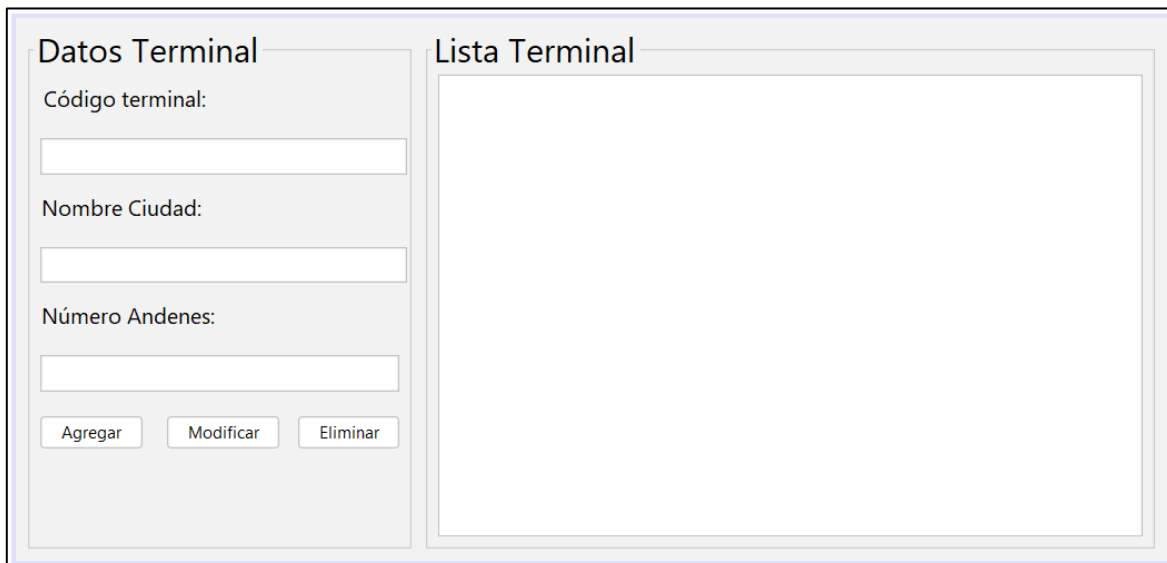
Una vez agregada, daremos clic derecho -> Table contents, se nos abrirá una ventana, seleccionamos Columns y seleccionamos todas las columnas, y las eliminamos:



En la pestaña de code del panel de propiedades, vamos a modificar todos los “variable name” de cada elemento para identificarlos a la hora de programar:



Y así quedaría el diseño del formulario:



Código

En el main de nuestra aplicación, que se encuentra en la clase CentralAutobus, escribiremos el siguiente código:

```
package com.mycompany.centralautobus;

public class CentralAutobus {

    public static void main(String[] args) {

        VistaTerminal objetoVista = new VistaTerminal();

        objetoVista.setVisible(true);

    }

}
```

Crearemos una clase llamada "ClaseTerminal" y vamos a escribir las siguientes variables y la función que va a mostrar las terminales en la tabla que agregamos.

```
package com.mycompany.centralautobus;

import java.sql.ResultSet;
import java.sql.Statement;
import javax.swing.JOptionPane;
import javax.swing.JTable;
import javax.swing.table.DefaultTableModel;

public class ClaseTerminal {

    private String codigoTerminal;
    private String nombreCiudad;
    private int numeroAndenes;

    public void MostrarTerminales(JTable tabTotEstaciones){

        ClaseConexion objetoConexion = new ClaseConexion();

        DefaultTableModel modelo= new DefaultTableModel();

        modelo.addColumn("Id");
```

```

        modelo.addColumn("Código Terminal");
        modelo.addColumn("Nombre ciudad");
        modelo.addColumn("Número Andenes");

        tabTotEstaciones.setModel(modelo);

        String sql = "SELECT * FROM Logistica.Terminal";
        Statement st;

        try{
            st = objetoConexion.establecerConexion().createStatement();
            ResultSet rs = st.executeQuery(sql);
            while(rs.next()){
                modelo.addRow(new Object[]{rs.getInt("IdTerminal"),
rs.getString("CodigoTerminal"),
                rs.getString("NombreCiudad"), rs.getInt("NumeroAndenes")});
            }
            tabTotEstaciones.setModel(modelo);
        }
        catch(Exception e){
            JOptionPane.showMessageDialog(null, "Error al cargar datos:
"+e.toString());
        }
    }
}

```

Dentro de la misma clase, crearemos la función que va a agregar a las terminales a la base de datos:

```

    public void insertarTerminal(JTextField codigo, JTextField nombreC, JTextField
andenesh){

```

```

        if(codigo.getText().trim().isEmpty() || nombreC.getText().trim().isEmpty() ||
        andenes.getText().trim().isEmpty()){

            JOptionPane.showMessageDialog(null, "Por favor, llena todos los campos
            antes de guardar.");

            return;

        }

        try {

            int ande = Integer.parseInt(andenes.getText().trim());

            this.codigoTerminal = codigo.getText().trim();

            this.nombreCiudad = nombreC.getText().trim();

            this.numeroAndenes = ande;

            ClaseConexion objetoConexion = new ClaseConexion();

            String consulta = "INSERT INTO Logistica.Terminal (CodigoTerminal,
            NombreCiudad, NumeroAndenes) VALUES (?, ?, ?)";

            CallableStatement cs =
            objetoConexion.establecerConexion().prepareCall(consulta);

            cs.setString(1, this.codigoTerminal);

            cs.setString(2, this.nombreCiudad);

            cs.setInt(3, this.numeroAndenes);

            cs.execute();

        } catch (NumberFormatException ex){

            JOptionPane.showMessageDialog(null, "El campo 'Andenes' solo debe
            contener números.", "Error de formato", JOptionPane.ERROR_MESSAGE);

        } catch (Exception e){

            JOptionPane.showMessageDialog(null, "Error en la base de datos: " +
            e.toString());

        }

    }
}

```

Agregamos la función para modificar:

```

public void modificarTerminal(int idTerminal, JTextField codigo, JTextField
nombreC, JTextField andenes){

```

```

int ande = Integer.parseInt(andenes.getText());
this.codigoTerminal = codigo.getText();
this.nombreCiudad = nombreC.getText();
this.numeroAndenes = ande;

ClaseConexion objetoConexion = new ClaseConexion();

String consulta = "UPDATE Logistica.Terminal SET CodigoTerminal = ?,
NombreCiudad = ?, NumeroAndenes = ? WHERE IdTerminal = ?";

try{
    CallableStatement cs =
objetoConexion.establecerConexion().prepareCall(consulta);
    cs.setString(1, this.codigoTerminal);
    cs.setString(2, this.nombreCiudad);
    cs.setInt(3, this.numeroAndenes);
    cs.setInt(4, idTerminal); // Usamos el ID directamente aquí
    cs.execute();
} catch(Exception e){
    JOptionPane.showMessageDialog(null, "Error al modificar: " + e.toString());
}
}

```

Agregamos la función para eliminar:

```

public void eliminarTerminal(int idTerminal){
    ClaseConexion objetoConexion = new ClaseConexion();

    String consulta = "DELETE FROM Logistica.Terminal WHERE IdTerminal =
?";

    try{
        CallableStatement cs =
objetoConexion.establecerConexion().prepareCall(consulta);

```



```

        cs.setInt(1, idTerminal);
        cs.execute();
    } catch (Exception e){
        JOptionPane.showMessageDialog(null, "Error al eliminar: " + e.toString());
    }
}

```

Nos dirigimos a nuestra “VistaTerminal”, seleccionamos el recuadro “Source” para acceder al código, y en el constructor, agregamos lo siguiente:

```

public VistaTerminal() {
    initComponents();

    ClaseTerminal objetoTerminal = new ClaseTerminal();
    try{
        objetoTerminal.MostrarTerminales(tbListaTerminal);
    }catch (Exception e){
        JOptionPane.showMessageDialog(null, "Error al cargaro datos: " +
e.toString());
    }
}

```

Volvemos al “Design” de la vista, daremos doble clic al botón de Agregar y agregaremos este código a su método:

```

private void btnAgregarActionPerformed(java.awt.event.ActionEvent evt) {
    ClaseTerminal objetoTerminal = new ClaseTerminal();
    try{
        objetoTerminal.insertarTerminal(txtCodTerm, txtNomCiu, txtNumAnd);
        objetoTerminal.MostrarTerminales(tbListaTerminal);
        txtCodTerm.setText("");
        txtNomCiu.setText("");
    }
}

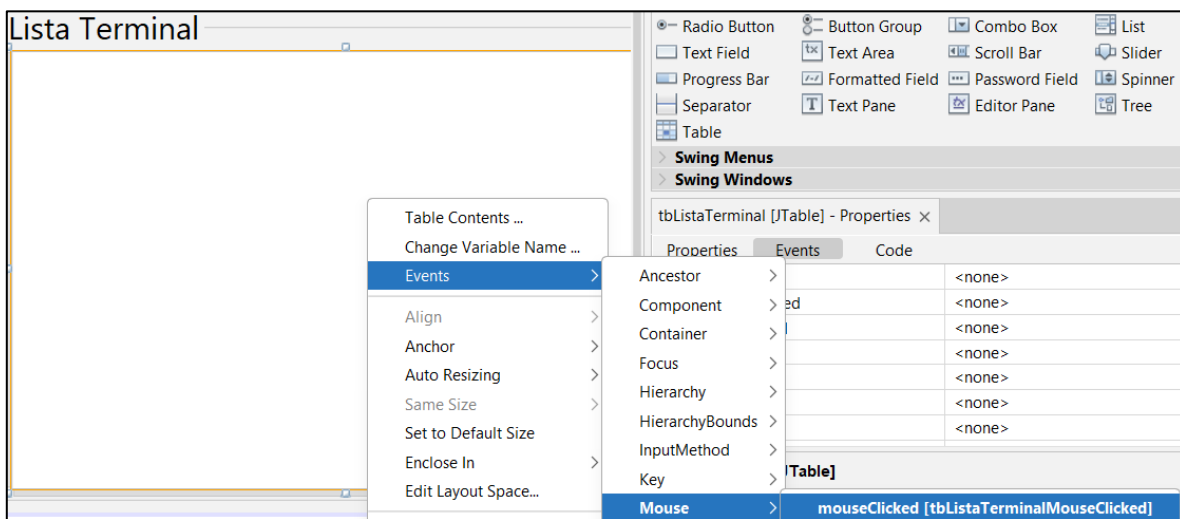
```

```

        txtNumAnd.setText("");
    }catch (Exception e){
        JOptionPane.showMessageDialog(null, "Error alinsertar los datos: " +
e.toString());
    }
}

```

Para cargar los elementos de la tabla en los campos, crearemos el siguiente evento (dándole clic derecho a la tabla de la lista de terminales -> Events -> MouseClicked):



Y agregaremos el siguiente código:

```

private void tbListaTerminalMouseClicked(java.awt.event.MouseEvent evt) {
    int filaSeleccionada = tbListaTerminal.getSelectedRow();
    if (filaSeleccionada >= 0) {
        // Solo subimos los datos editables a los TextFields (columnas 1, 2 y 3)
        txtCodTerm.setText(tbListaTerminal.getValueAt(filaSeleccionada,
1).toString());
        txtNomCiu.setText(tbListaTerminal.getValueAt(filaSeleccionada,
2).toString());
        txtNumAnd.setText(tbListaTerminal.getValueAt(filaSeleccionada,
3).toString());
    }
}

```

Daremos doble clic al botón de Modificar y agregamos este código:

```
private void btnModificarActionPerformed(java.awt.event.ActionEvent evt) {  
    int filaSeleccionada = tbListaTerminal.getSelectedRow();  
    if (filaSeleccionada >= 0) {  
        int idDesdeTabla =  
Integer.parseInt(tbListaTerminal.getValueAt(filaSeleccionada, 0).toString());  
        ClaseTerminal objetoTerminal = new ClaseTerminal();  
        try {  
            objetoTerminal.modificarTerminal(idDesdeTabla, txtCodTerm, txtNomCiu,  
txtNumAnd);  
            objetoTerminal.MostrarTerminales(tbListaTerminal);  
            txtCodTerm.setText("");  
            txtNomCiu.setText("");  
            txtNumAnd.setText("");  
        } catch (Exception e){  
            JOptionPane.showMessageDialog(null, "Error al modificar los datos: " +  
e.toString());  
        }  
    } else {  
        JOptionPane.showMessageDialog(null, "Por favor, selecciona un renglón  
de la tabla para modificar.");  
    }  
}
```

Y finalmente con el botón de eliminar:

```
private void btnEliminarActionPerformed(java.awt.event.ActionEvent evt) {  
    int filaSeleccionada = tbListaTerminal.getSelectedRow();  
    if (filaSeleccionada >= 0) {  
        int idDesdeTabla =  
Integer.parseInt(tbListaTerminal.getValueAt(filaSeleccionada, 0).toString());
```

```

ClaseTerminal objetoTerminal = new ClaseTerminal();
try {
    objetoTerminal.eliminarTerminal(idDesdeTabla);
    objetoTerminal.MostrarTerminales(tbListaTerminal);
    txtCodTerm.setText("");
    txtNomCiu.setText("");
    txtNumAnd.setText("");
} catch (Exception e){
    JOptionPane.showMessageDialog(null, "Error en el proceso de
eliminación: " + e.toString());
}
}
else {
    JOptionPane.showMessageDialog(null, "Por favor, selecciona un renglón
de la tabla para eliminar.");
}
}
}

```

Y así quedaría nuestro formulario:

Id	Código Terminal	Nombre ciudad	Número Andenes
1	Terminal 1	Ciudad 1	100

Script completo de la base de datos

-- Database: CentralAutobus

--1 Creación de la base de datos

CREATE DATABASE "CentralAutobus"

WITH

OWNER = postgres

ENCODING = 'UTF8'

LOCALE_PROVIDER = 'libc'

CONNECTION LIMIT = -1

IS_TEMPLATE = False;

--2 Creación de esquemas

CREATE SCHEMA Atencion_Cliente;

CREATE SCHEMA Logistica;

--3 Creación de Tablas

CREATE TABLE Atencion_Cliente.Pasajero

(

IdPasajero BIGSERIAL NOT NULL,

NombrePasajero VARCHAR(150) NOT NULL,

Sexo VARCHAR(15) NOT NULL,

DocumentoIdentidad VARCHAR(20) NOT NULL,

TotalEquipaje INT,

CONSTRAINT PK_PASAJERO PRIMARY KEY(IdPasajero)

);

```

CREATE TABLE Atencion_Cliente.Equipaje
(
    IdEquipaje BIGSERIAL NOT NULL,
    IdPasajero BIGINT NOT NULL,
    TipoEquipaje VARCHAR(50) NOT NULL,
    PesoKg DECIMAL (5,2) NOT NULL,

    CONSTRAINT PK_EQUIPAJE PRIMARY KEY(IdEquipaje),
    CONSTRAINT FK_EQUIPAJE_PASAJERO FOREIGN KEY(IdPasajero)
    REFERENCES Atencion_Cliente.Pasajero(IdPasajero),
    CONSTRAINT CHK_PESO_EQUIPAJE CHECK (PesoKg >= 0.0)
);

```

```

CREATE TABLE Logistica.Terminal
(
    IdTerminal BIGSERIAL NOT NULL,
    CodigoTerminal VARCHAR(10) NOT NULL,
    NombreCiudad VARCHAR(100) NOT NULL,
    NumeroAndenes INT NOT NULL,

    CONSTRAINT PK_TERMINAL PRIMARY KEY(IdTerminal),
    CONSTRAINT UQ_CODIGO_TERMINAL UNIQUE(CodigoTerminal)
);

```

```

CREATE TABLE Logistica.Boleto
(
    IdBoleto BIGSERIAL NOT NULL,

```

```

IdPasajero BIGINT NOT NULL,
IdTerminalOrigen BIGINT NOT NULL,
IdTerminalDestino BIGINT NOT NULL,
FechaSalida TIMESTAMP NOT NULL,
TotalPago MONEY,

CONSTRAINT PK_BOLETO PRIMARY KEY(IdBoleto),
CONSTRAINT FK_BOLETO_PASAJERO FOREIGN KEY(IdPasajero)
REFERENCES Atencion_Cliente.Pasajero(IdPasajero),
CONSTRAINT FK_BOLETO_ORIGEN FOREIGN KEY(IdTerminalOrigen)
REFERENCES Logistica.Terminal(IdTerminal),
CONSTRAINT FK_BOLETO_DESTINO FOREIGN KEY(IdTerminalDestino)
REFERENCES Logistica.Terminal(IdTerminal)
);

```

--4 Creación de la función del Trigger

```

CREATE OR REPLACE FUNCTION Atencion_Cliente.fn_recargo equipaje()
RETURNS TRIGGER AS $$ DECLARE
    v_peso_total DECIMAL(5,2);
    v_maletas INT;
    v_recargo DECIMAL(10,2):=0;
BEGIN
    --Cálculo de totales para el pasajeo
    SELECT COUNT(*), SUM(PesoKg)
    INTO v_maletas, v_peso_total FROM Atencion_Cliente.Equipaje
    WHERE IdPasajero = NEW.IdPasajero;
    --Aplicar las reglas
    IF v_peso_total > 25.00 THEN

```

```

        v_recargo := 1000.00;
ELSIF v_maletas > 1 THEN
        v_recargo := 1000.00;
END IF;
--Actualización del Total del pago
UPDATE Logistica.Boleto
SET TotalPago = TotalPago + v_recargo::NUMERIC::MONEY
WHERE IdBoleto = (
        SELECT IdBoleto
        FROM Logistica.Boleto
        WHERE IdPasajero = NEW.IdPasajero
        ORDER BY FechaSalida DESC
        LIMIT 1
);
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

--5 Disparador
CREATE TRIGGER trg_recargo_equipaje
AFTER INSERT OR UPDATE ON Atencion_Cliente.Equipaje
FOR EACH ROW EXECUTE FUNCTION Atencion_Cliente.fn_recargo_equipaje();

--Creacion de Usuarios

--6 Creacion de roles
CREATE ROLE Capturista;
CREATE ROLE Vendedor;

```


--7 Creacion de usuarios

```
CREATE USER Emmanuel_Loredo WITH PASSWORD 'Emma123';
```

```
CREATE USER Manuel_Morales WITH PASSWORD 'Mane456';
```

--8 Asignacion de roles

```
GRANT Capturista TO Emmanuel_Loredo;
```

```
GRANT Vendedor TO Manuel_Morales;
```

--9 Permisos a la BD y esquemas

```
GRANT CONNECT ON DATABASE "CentralAutobus" TO Capturista, Vendedor;
```

```
GRANT USAGE ON SCHEMA Atencion_Cliente TO Capturista;
```

```
GRANT USAGE ON SCHEMA Logistica TO Vendedor;
```

```
GRANT USAGE ON ALL SEQUENCES IN SCHEMA Atencion_Cliente TO  
Capturista;
```

--Capturista permisos

```
GRANT SELECT, INSERT ON Atencion_Cliente.Pasajero TO Capturista;
```

--Vendedor permisos

```
GRANT SELECT ON Logistica.Boleto TO Vendedor;
```

```
REVOKE INSERT ON Logistica.Boleto FROM Vendedor;
```